Content Opportunity Scoring: A Data-Driven Framework for Prioritizing Search Content Review

FlyRank ML Internship — Capstone Research

Abstract

Content teams need a repeatable way to decide which pages deserve review when a large portfolio cannot be inspected equally. This study asks whether historical search-performance and query-mix signals can identify pages associated with a subsequent decline in impressions. Using the FlyRank full pseudonymized warehouse, the analysis aggregates daily content performance into 30-day historical features and combines them with query-level signals before fitting a 200-tree random forest. A client-grouped holdout evaluation produced 0.938 accuracy and 0.822 macro F1 for the random forest, compared with a 0.875 accuracy and approximately 0.466 macro F1 for a majority-class baseline. The resulting framework is presented as a ranked content-review decision aid rather than a causal explanation of search-engine rankings or a guarantee that changing a page will improve performance.

1. Introduction / Problem Statement

Content teams often have more pages to review than they can realistically inspect in the same cycle. This capstone addresses the decision: Which content pages should be prioritized for review based on observable historical search-performance signals? The chosen lane is Refresh / Content Opportunity Scoring. The goal is to build a repeatable ranking-oriented workflow that converts historical data into an actionable review queue, without claiming to prove why a search engine changed a page's visibility.

2. Data

The analysis uses the FlyRank full pseudonymized warehouse release. The supplied Week 3+ notebook describes approximately 17 months of daily search-performance history, an unbalanced client panel, and a query-level table. The executed connection contained 78,835,655 rows in fact_content_daily_performance, 11,694,072 rows in the sample daily table, and 2,414,248 rows in fact_content_query_90d. The connected dimensions included dim_clients and dim_content.

2.1 Release and tables

The feature pipeline used fact_content_daily_performance for daily impressions, clicks, average position, and historical position volatility; fact_content_query_90d for visible-query count, rare-impression share, anonymized-impression share, and query concentration; and dim_clients for client-grouped validation. No client names, domains, URLs, private queries, credentials, or raw exports are published.

2.2 Analysis window and eligibility

The full-release feature query determines the latest available report_date and works over the preceding 90 days. It creates a previous-30-day feature window and a last-30-day outcome window. Content items are retained when previous-30-day impressions are at least 200. The executed feature query produced 114,435 eligible content items, and the query-signal join retained 114,435 rows. Missing values in the six model features were excluded before fitting.

3. Methodology

Features were engineered inside DuckDB so the large Parquet warehouse did not need to be loaded into memory. The model used six pre-outcome signals: imp_prev30, visible_queries, rare_share, anon_share, top_query_share, and pos_volatility_prev30. The binary label is is_declining = 1 when last-30-day impressions are below 80% of previous-30-day impressions. The proposed model is a RandomForestClassifier with 200 trees and random state 42. The baseline is a majority-class classifier.

3.1 Validation and leakage

The headline evaluation uses GroupShuffleSplit with client_hash_id as the grouping variable, a 25% test fraction, and random state 42. This prevents pages from the same client appearing in both training and evaluation. The notebook also demonstrates leakage: trend_pct is derived from the same trend outcome as the label, so it is excluded from the full-release model. The reported model instead uses historical and query-mix signals available before the outcome window.

4. Results

On the client-grouped holdout, the random forest achieved 0.938 accuracy, 0.822 macro F1, and 0.930 weighted F1. The majority baseline achieved 0.875 accuracy, approximately 0.466 macro F1, and approximately 0.816 weighted F1. The model therefore improved accuracy by 6.3 percentage points and substantially improved macro F1 on the same holdout.

4.1 Class-level performance

For the non-declining class, precision was 0.956, recall 0.527, and F1 0.679 across 1,556 examples. For the declining class, precision was 0.936, recall 0.997, and F1 0.966 across 10,879 examples. The high declining-class recall is useful for review prioritization, but false positives remain, so the output should not automatically trigger content changes.

4.2 Feature importance

The fitted random forest reported the following feature importances: imp_prev30 0.215; anon_share 0.190; pos_volatility_prev30 0.165; rare_share 0.160; visible_queries 0.135; and top_query_share 0.134. These are predictive importance measures, not causal effects or directional recommendations.

5. Ranked Recommendations

The validated model supports a practical review queue. Because the supplied full-release notebook does not export a public-safe final page-level prediction table from the grouped-holdout model, this paper does not fabricate page rankings or expose raw page identifiers. The ranked action playbook is: (1) review high-exposure pages flagged as declining; (2) inspect pages with unstable historical positions; (3) review query-mix dependence and concentration; (4) inspect rare-query exposure; (5) review query breadth; and (6) protect or monitor pages without strong decline evidence. Model probability should determine page ordering in the final notebook, while the action playbook determines the human review response.

6. Limitations & Honest Framing

This analysis is associative rather than causal. It does not prove Google's ranking algorithm, ranking factors, algorithm updates, or causal refresh effects. The headline validation tests client generalization rather than temporal generalization. Class imbalance makes accuracy alone insufficient, so macro F1 is also reported. Random-forest feature importance describes model usage, not whether increasing or decreasing a signal is beneficial. Public reporting intentionally omits client names, domains, URLs, private queries, credentials, and raw exports.

7. Reproducibility

The work is based on the three supplied internship notebooks. Notebook 01 establishes the refresh-ranking problem and starter workflow; Notebook 02 demonstrates readable modeling and leakage; Notebook 03 moves to the full pseudonymized warehouse, aggregates it with DuckDB, joins query-level signals, trains the random forest, and performs client-grouped validation. Repository: https://github.com/vishal6975/flyrank-ml-internship

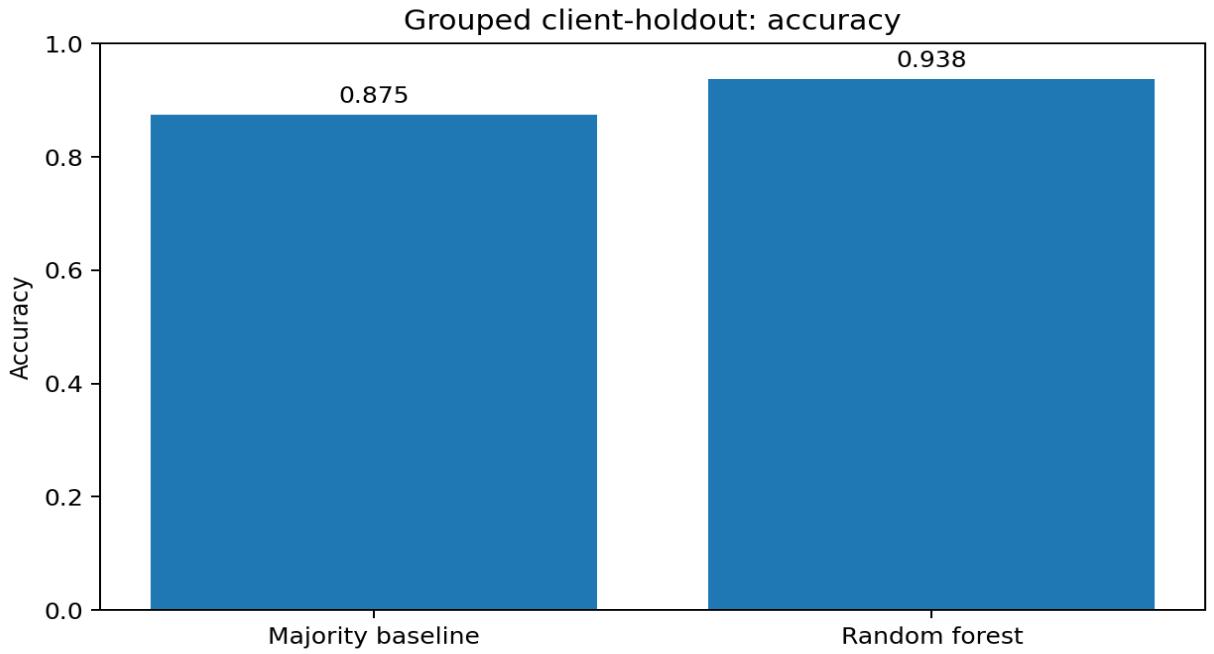
8. Acknowledgments & Data Credit

Built on the FlyRank ML Internship dataset. Data source: https://flyrank.ai. The analysis follows the internship public-safe requirements.

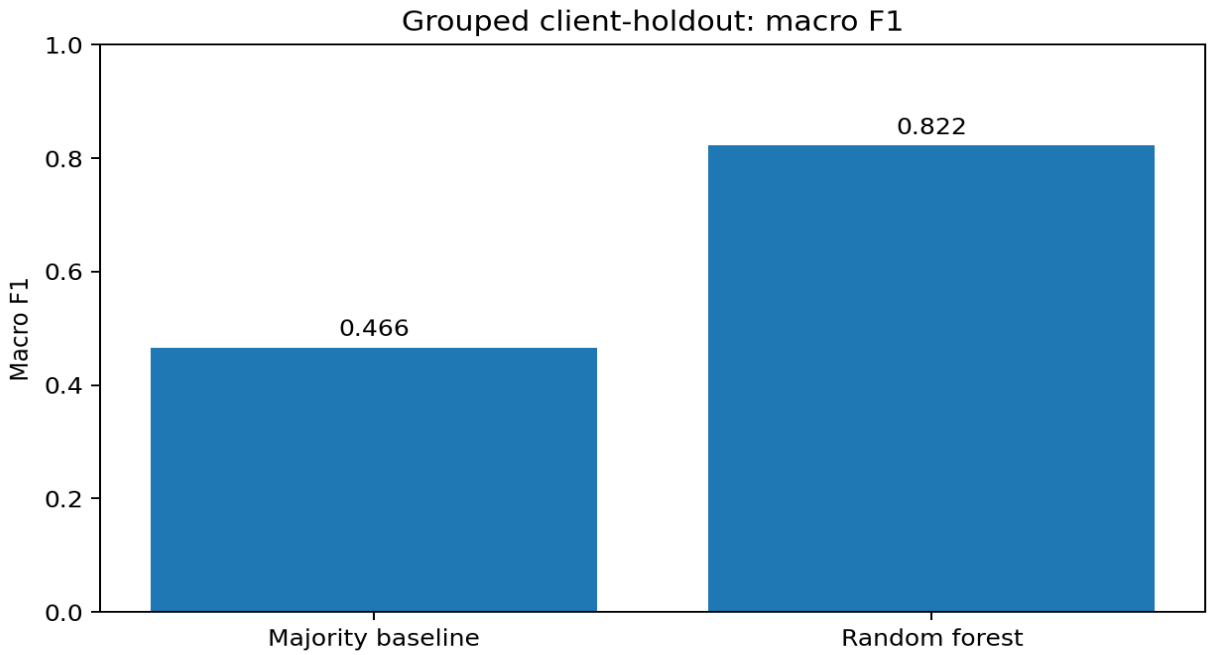
9. Conclusion

This study demonstrates a repeatable approach for prioritizing content review from real search-performance data. On the executed full-release workflow, a 200-tree random forest using six pre-outcome performance and query-mix signals achieved 0.938 accuracy and 0.822 macro F1 on a client-grouped holdout, outperforming a majority-class baseline on the same split. The practical output is a model-informed review queue for human investigation, not an automated content-change system or causal explanation of search behavior. A future extension should add strict temporal backtesting and a public-safe prediction export to make the ranked page-level queue directly reproducible.

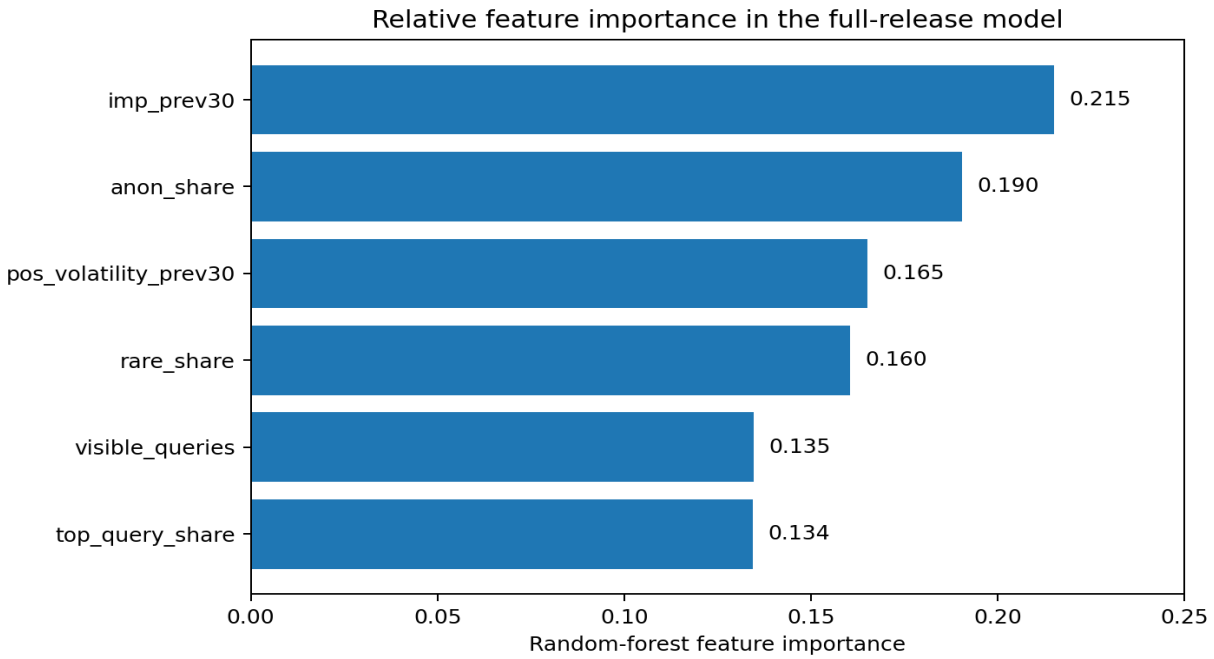
Figures



Accuracy on the client-grouped holdout.



Macro F1 on the client-grouped holdout.



Random-forest feature importances.